

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Modular Code Structure Project Report

A detailed 20-page academic-style report covering Brainstorming and Ideation, Requirement Analysis, Project Design, Project Planning, Development, Testing, Documentation and Demonstration.

Project focus: smart budgeting, expense analysis, savings goals, explainable recommendations, privacy-aware AI assistance and maintainable modular software architecture.

Prepared as a structured software project document.

1. 1. Brainstorming and Ideation

Focus: Problem definition, user pain points, feature discovery, MVP, modular thinking.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on problem definition, user pain points, feature discovery, mvp, modular thinking.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

2. 2. Brainstorming and Ideation

Focus: Feature prioritization, personas, use cases, recommendation concept, scope boundaries.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on feature prioritization, personas, use cases, recommendation concept, scope boundaries.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

3. 3. Requirement Analysis

Focus: Functional requirements, transaction management, budgets, goals, dashboard and recommendations.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on functional requirements, transaction management, budgets, goals, dashboard and recommendations.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

4. 4. Requirement Analysis

Focus: Non-functional requirements, security, privacy, performance, usability, reliability and traceability.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on non-functional requirements, security, privacy, performance, usability, reliability and traceability.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

5. 5. Project Design

Focus: Layered architecture, module responsibilities, dependency direction and system workflow.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on layered architecture, module responsibilities, dependency direction and system workflow.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

6. 6. Project Design

Focus: Database entities, API boundaries, UI structure, data validation and modular directory layout.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on database entities, api boundaries, ui structure, data validation and modular directory layout.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Suggested directory structure

```
pocketsmart/  
  app/  
    main.py  
    config.py  
  api/  
    routes_transactions.py  
    routes_budgets.py  
    routes_goals.py  
    routes_recommendations.py  
  domain/  
    models.py  
    schemas.py  
  services/  
    transaction_service.py  
    budget_service.py  
    goal_service.py  
    recommendation_service.py  
  analytics/  
    spending.py  
    cashflow.py  
    repositories/
```

```
transaction_repository.py
budget_repository.py
goal_repository.py
recommendations/
rules.py
explanations.py
tests/
unit/
integration/
e2e/
```

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

7. 7. Project Planning

Focus: Work breakdown, milestones, development sequence, resources, version control and review process.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on work breakdown, milestones, development sequence, resources, version control and review process.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

8. 8. Project Planning

Focus: Risk management, quality gates, synthetic demonstration data, change control and delivery plan.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on risk management, quality gates, synthetic demonstration data, change control and delivery plan.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

9. 9. Development

Focus: Environment setup, FastAPI-style application bootstrap, configuration, dependencies and coding standards.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on environment setup, fastapi-style application bootstrap, configuration, dependencies and coding standards.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

10. 10. Development

Focus: Transaction, category and budget modules with sample modular service code.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on transaction, category and budget modules with sample modular service code.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Sample modular service

```
class BudgetService:
    def __init__(self, budget_repo, transaction_repo):
        self.budget_repo = budget_repo
        self.transaction_repo = transaction_repo

    def utilization(self, user_id, category, start, end):
        budget = self.budget_repo.get(user_id, category, start, end)
        spent = self.transaction_repo.sum_expenses(
            user_id, category, start, end
        )
        if not budget or budget.limit <= 0:
            return {"limit": 0, "spent": spent, "percent": 0}
        return {"limit": budget.limit,
            "spent": spent,
            "percent": (spent / budget.limit) * 100}
```

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

11. 11. Development

Focus: Savings goals, analytics, monetary calculations, repositories and integration boundaries.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on savings goals, analytics, monetary calculations, repositories and integration boundaries.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

12. 12. Development

Focus: AI recommendation pipeline, deterministic metrics, rules, explanations and optional AI phrasing.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on ai recommendation pipeline, deterministic metrics, rules, explanations and optional ai phrasing.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

13. 13. Development

Focus: Security, privacy, controlled AI tools, error handling, logging and deployment considerations.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on security, privacy, controlled ai tools, error handling, logging and deployment considerations.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

14. 14. Testing

Focus: Testing strategy, unit tests, validation, calculation tests, integration and API testing.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on testing strategy, unit tests, validation, calculation tests, integration and api testing.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Example unit test

```
def test_budget_utilization():
    service = BudgetService(FakeBudgetRepo(1000),
                             FakeTransactionRepo(250))
    result = service.utilization("u1", "food",
                                 "2026-09-01", "2026-09-30")
    assert result["spent"] == 250
    assert result["percent"] == 25
```

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

15. 15. Testing

Focus: End-to-end testing, edge cases, authorization, regression testing, acceptance criteria and quality evidence.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on end-to-end testing, edge cases, authorization, regression testing, acceptance criteria and quality evidence.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

16. 16. Documentation

Focus: README, API documentation, code documentation, database documentation and user guide.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on readme, api documentation, code documentation, database documentation and user guide.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

17. 17. Documentation

Focus: Deployment guide, troubleshooting, maintenance, change log, limitations and responsible AI notes.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on deployment guide, troubleshooting, maintenance, change log, limitations and responsible ai notes.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

18. 18. Demonstration

Focus: Demo preparation, synthetic data, step-by-step workflow, screenshots, code walkthrough and evidence.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on demo preparation, synthetic data, step-by-step workflow, screenshots, code walkthrough and evidence.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

19. 19. Demonstration

Focus: Presentation script, expected outputs, evaluator questions, architecture explanation and future extensions.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on presentation script, expected outputs, evaluator questions, architecture explanation and future extensions.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.

20. 20. Final Integration and Conclusion

Focus: Complete workflow, maintainability benefits, future enhancements, project outcomes and conclusion.

Development follows the modular structure rather than placing all logic in one application file. Routes remain thin, services implement workflows, repositories manage persistence, analytics perform deterministic calculations, and recommendation modules evaluate explicit rules. This makes the application easier to test, extend and explain.

This section focuses specifically on complete workflow, maintainability benefits, future enhancements, project outcomes and conclusion.. The objective is to turn the project concept into concrete engineering work that can be implemented, reviewed and demonstrated.

The application should maintain a clear boundary between user interface concerns and business rules. A form may collect an amount, category and date, but it should not calculate budget utilization or write directly to a database. Instead, the request is validated and passed to an application service that owns the relevant workflow.

The modular code structure is important because each major responsibility changes at a different rate. Database schemas may evolve, recommendation rules may be refined, dashboards may be redesigned, and AI wording may be replaced. Keeping these responsibilities separate reduces the amount of unrelated code that must be changed for one improvement.

PocketSmart AI should treat numerical financial facts as deterministic application data. Totals, category spending, budget utilization and savings progress should be calculated by tested application logic. If an AI model is used, it should receive approved metrics and help communicate them rather than inventing balances or transaction values.

Validation should occur at system boundaries. Required fields, supported categories, dates, monetary amounts and ownership identifiers must be checked before processing. Important constraints should also be enforced at the persistence layer so that invalid data cannot be introduced by an alternate code path.

The design should support synthetic test data, repeatable automated tests and a reproducible demonstration. This is especially useful for an academic project because the evaluator can see the same workflow repeatedly without exposing real personal financial information.

Security and privacy should be treated as design requirements rather than final-stage additions. User-owned records must be protected with authentication and authorization, secrets should be outside source code, logs should avoid unnecessary financial details, and external AI services should receive only the minimum data required for an authorized task.

The project should maintain traceability from requirement to implementation and test. For example, a budget requirement maps to budget routes, budget services, repository operations, utilization calculations, interface elements and test cases. This makes the final report more credible because each major claim can be demonstrated.

A useful implementation rule is to prefer small functions with clear inputs and outputs. Pure calculation functions are particularly valuable because they can be tested without starting the web server or connecting to a live database.

Error handling should be predictable. Invalid user input should result in a useful validation response; unexpected server failures should be logged internally without exposing stack traces or sensitive implementation details to the user.

The user interface should explain results in simple language. A recommendation card can show the observation, the period, the category, the supporting metric and an optional action for the user to consider. This gives the user enough context to evaluate the recommendation independently.

The final implementation should be reviewed against the original scope. Optional features such as receipt extraction, forecasting, conversational queries or multi-currency support should not destabilize the minimum viable product. They can be documented as future enhancements.

Overall, the section contributes to a disciplined development process in which requirements, architecture, modular code, tests, documentation and demonstration evidence remain connected.

Implementation checkpoint: keep this responsibility inside its designated module.

Quality checkpoint: add a deterministic test for important calculations and boundary conditions.

Traceability checkpoint: connect the requirement, implementation component, test evidence and demonstration step.

Maintainability checkpoint: avoid direct dependencies on unrelated modules.

Security checkpoint: protect user-owned records and keep sensitive configuration outside source control.

Documentation checkpoint: update the relevant explanation whenever an interface or business rule changes.